MECH 650 - Assignment 1

American University of Beirut

Prof. Daniel Asmar

Suhaib Abu-Raidah, Boulos Boulos, Daniel Haikal

Exercise 1.1

Differential Robot

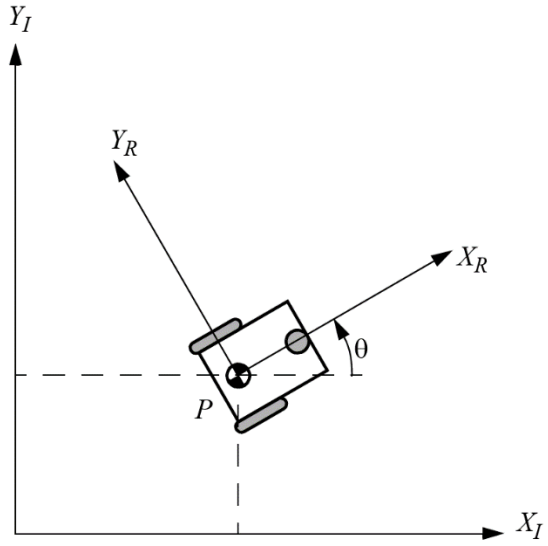

Figure 1: Reference frame and parameters of the differential drive robot

To solve for the kinematics of the differential robot, we should set the rolling and sliding constraints for each wheel that is either powered or impose constraint on the motion of the robot.

The only wheels accounted for are the two fixed standard wheels. The castor wheel is neglected because it's not powered nor does it impose any motion constraints.

Rolling Constraints:

The rolling constraint equation is as follows:

$$j_1(B_s)R(\theta)\dot{\zeta}_I - J_2\dot{\phi} = 0 \qquad (1)$$

The wheels parameters are defined according to *Figure 1*:

The wheel parameters for the first wheel:

$$\alpha_1 = \frac{\pi}{2}, \qquad \beta_1 = 0, \qquad r = r_1$$

The wheel parameters for the second wheel:

$$\alpha_2 = -\frac{\pi}{2}, \qquad \beta_2 = \pi, \qquad r = r_2$$

Substituting for the two fixed standard wheels in equation (1):

$$\begin{bmatrix} \sin(\alpha_1+\beta_1) & -\cos(\alpha_1+\beta_1) & -l\cos(\beta_1) \\ \sin(\alpha_2+\beta_2) & -\cos(\alpha_2+\beta_2) & -l\cos(\beta_2) \end{bmatrix} R(\theta)\dot{\zeta}_I - \begin{bmatrix} r_1 & 0 \\ 0 & r_2 \end{bmatrix} \begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \end{bmatrix} = 0 \qquad (2)$$

Substituting the values of αs and βs in equation (2), we get:

$$\begin{bmatrix} 1 & 0 & -l \\ 1 & 0 & l \end{bmatrix} R(\theta)\dot{\zeta}_I - \begin{bmatrix} r_1 & 0 \\ 0 & r_2 \end{bmatrix} \begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \end{bmatrix} = 0 \qquad (3)$$

Sliding Constraints:

The sliding constraint equation is as follows:

$$C_1(B_s)R(\theta)\dot{\zeta}_I = 0 \qquad (4)$$

Substituting for the two fixed standard wheels in equation (4):

$$\begin{bmatrix} \cos(\alpha_1+\beta_1) & \sin(\alpha_1+\beta_1) & l\sin(\beta_1) \\ \cos(\alpha_2+\beta_2) & \sin(\alpha_2+\beta_2) & l\sin(\beta_2) \end{bmatrix} R(\theta)\dot{\zeta}_I = 0 \qquad (5)$$

Substituting the values of αs and βs in equation (5), we get:

$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix} R(\theta)\dot{\zeta}_I = 0 \tag{6}$$

Since the two rows of $C_1(B_s)$ in equation (6) are not independent, we can omit one of them. The equation becomes:

$$[0 \quad 1 \quad 0]R(\theta)\dot{\zeta}_I = 0 \tag{7}$$

Combining equation (3) and equation (7), we get:

$$\begin{bmatrix} 1 & 0 & -l \\ 1 & 0 & l \\ 0 & 1 & 0 \end{bmatrix} R(\theta)\dot{\zeta}_I = \begin{bmatrix} r_1\dot{\phi}_1 \\ r_2\dot{\phi}_2 \\ 0 \end{bmatrix} \tag{8}$$

Solving for $\dot{\zeta}_I$ from equation (8):

$$\dot{\zeta}_I = R^{-1}(\theta)\begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 \\ \frac{1}{2l} & -\frac{1}{2l} & 0 \end{bmatrix}\begin{bmatrix} r_1\dot{\phi}_1 \\ r_2\dot{\phi}_2 \\ 0 \end{bmatrix} \tag{9}$$

Omni-drive Robot

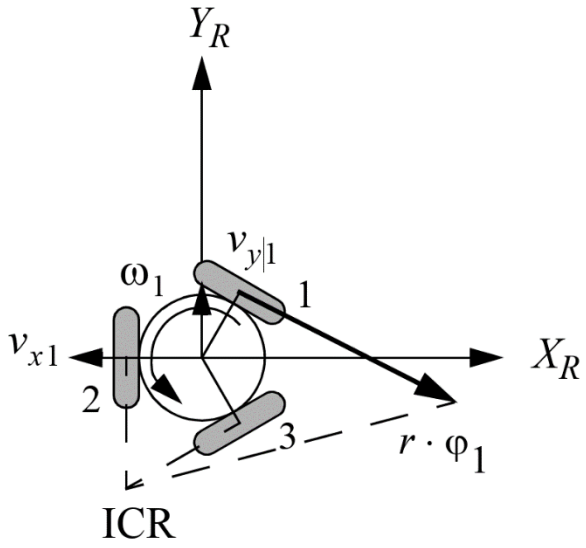

Figure 2: Reference frame and parameters of the Omni-drive robot defined.

For the Omni-drive robot, all of the three Swedish wheels are assumed to be powered, therefore they are all accounted for in the kinematics.

Rolling Constraints:

The wheels parameters are defined according to Figure 2:

The wheel parameters for the first wheel:

$$\alpha_1 = \frac{\pi}{3}, \qquad \beta_1 = 0, \qquad \gamma_1 = 0, \qquad r = r_1$$

The wheel parameters for the second wheel:

$$\alpha_2 = \pi, \qquad \beta_2 = 0, \qquad \gamma_2 = 0, \qquad r = r_2$$

The wheel parameters for the second wheel:

$$\alpha_3 = -\frac{\pi}{3}, \qquad \beta_3 = 0, \qquad \gamma_3 = 0, \qquad r = r_3$$

Substituting for the three Swedish wheels in equation (1):

$$\begin{bmatrix} \sin(\alpha_1+\beta_1+\gamma_1) & -\cos(\alpha_1+\beta_1+\gamma_1) & -l\cos(\beta_1+\gamma_1) \\ \sin(\alpha_2+\beta_2+\gamma_2) & -\cos(\alpha_2+\beta_2+\gamma_2) & -l\cos(\beta_2+\gamma_2) \\ \sin(\alpha_3+\beta_3+\gamma_3) & -\cos(\alpha_3+\beta_3+\gamma_3) & -l\cos(\beta_3+\gamma_3) \end{bmatrix} R(\theta)\dot{\zeta}_I - \begin{bmatrix} \cos(\gamma_1)\, r_1 & 0 & 0 \\ 0 & \cos(\gamma_2)\, r_2 & 0 \\ 0 & 0 & \cos(\gamma_3)\, r_3 \end{bmatrix} \begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \\ \dot{\phi}_3 \end{bmatrix} = 0 \tag{10}$$

Substituting the values of αs and βs in equation (10), we get:

$$\begin{bmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} & -l \\ 0 & 1 & -l \\ -\frac{\sqrt{3}}{2} & -\frac{1}{2} & -l \end{bmatrix} R(\theta)\dot{\zeta}_I - \begin{bmatrix} r_1 & 0 & 0 \\ 0 & r_2 & 0 \\ 0 & 0 & r_3 \end{bmatrix} \begin{bmatrix} \dot{\phi}_1 \\ \dot{\phi}_2 \\ \dot{\phi}_3 \end{bmatrix} = 0 \tag{11}$$

Solving for $\dot{\zeta}_I$ from equation (11):

$$\dot{\zeta}_I = R^{-1}(\theta) \begin{bmatrix} \frac{1}{\sqrt{3}} & 0 & -\frac{1}{\sqrt{3}} \\ -\frac{1}{3} & \frac{2}{3} & -\frac{1}{3} \\ -\frac{1}{3l} & -\frac{1}{3l} & -\frac{1}{3l} \end{bmatrix} \begin{bmatrix} r_1\dot{\phi}_1 \\ r_2\dot{\phi}_2 \\ r_3\dot{\phi}_3 \end{bmatrix} \tag{12}$$

Exercise 1.1.1

a)

The equation governing the differential drive robot kinematics from the previous exercise is as follows:

$$\dot{\zeta}_I = R^{-1}(\theta)\begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 \\ \frac{1}{2l} & -\frac{1}{2l} & 0 \end{bmatrix}\begin{bmatrix} r_1\dot{\phi}_1 \\ r_2\dot{\phi}_2 \\ 0 \end{bmatrix} \tag{13}$$

Substituting for $R^{-1}(\theta)$:

$$\dot{\zeta}_I = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 0 & 1 \\ \frac{1}{2l} & -\frac{1}{2l} & 0 \end{bmatrix}\begin{bmatrix} r_1\dot{\phi}_1 \\ r_2\dot{\phi}_2 \\ 0 \end{bmatrix} \tag{14}$$

By simplifying equation (14) and replacing θ with q, we get:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{q} \end{bmatrix} = \begin{bmatrix} \frac{1}{2}\left(r_1\dot{\phi}_1 + r_2\dot{\phi}_2\right)\cos q \\ \frac{1}{2}\left(r_1\dot{\phi}_1 + r_2\dot{\phi}_2\right)\sin q \\ \frac{1}{2l}\left(r_1\dot{\phi}_1 - r_2\dot{\phi}_2\right) \end{bmatrix} \tag{15}$$

To solve equation (15) we need to do incremental iteration in time, where each iteration is after a step Δt.

Let:

$\Delta\phi_1 = \dot{\phi}_1\Delta t$

$\Delta\phi_2 = \dot{\phi}_2\Delta t$

$\Delta q = \frac{\mathrm{r}_1\Delta\phi_1 - r_2\Delta\phi_2}{2l}$

Discretizing the integration of equation (15) and using the notation above, we get:

$$x_{k+1} = x_k + \frac{1}{2}(r_1\Delta\phi_1 + r_2\Delta\phi_2)\cos\left(q_k + \frac{\Delta q}{2}\right) \tag{16}$$

$$y_{k+1} = y_k + \frac{1}{2}(r_1\Delta\phi_1 + r_2\Delta\phi_2)\sin\left(q_k + \frac{\Delta q}{2}\right) \tag{17}$$

$$q_{k+1} = q_k + \Delta q \tag{18}$$

b)

Influence of the sampling time Δt on odometry precision:

The sampling time Δt directly affects the accuracy of the numerical integration used to estimate the robot pose. For large Δt we get larger integration errors and position and orientation errors grow faster with time. For small Δt, we get better accuracy and approximation, however it is more expensive computationally.

Relation between wheel speeds $(\dot{\phi}_1, \dot{\phi}_2)$ and robot velocities (v, ω):

The velocity of the robot (v) is directly proportional to both $(\dot{\phi}_1, \dot{\phi}_2)$, increase in either of them increase (v) and vice versa.

$$v = \frac{1}{2}\left(r_1\dot{\phi}_1 + r_2\dot{\phi}_2\right) \tag{19}$$

In the other hand the angular speed of the robot (ω) is directly proportional with difference between $\left(r_1\dot{\phi}_1, r_2\dot{\phi}_2\right)$. The increase in the difference increases (ω) and vice versa.

$$\omega = \frac{1}{2l}\left(r_1\dot{\phi}_1 - r_2\dot{\phi}_2\right) \tag{20}$$

Exercise 2

a) For both cases, we define the kinematic constraints on the wheels:

$$\begin{pmatrix} \sin\alpha+\beta & -\cos\alpha+\beta & -l\cos\beta \\ \cos\alpha+\beta & \sin\alpha+\beta & -l\cos\beta \end{pmatrix} \dot{\zeta}_R = \begin{pmatrix} r\dot{\phi} \\ 0 \end{pmatrix}$$

In the case of our diff-drive robot, we can replace α and β by the corresponding angle for each wheel to solve for motor velocities.

For the right wheel, (α,β) can be substituted by $(-\frac{\pi}{2}, \pi)$, which yields the equations:

$$\begin{cases} \dot{x} + l\dot{\theta} = r\dot{\phi}_r \\ \dot{y} = 0 \end{cases}$$

For the left wheel, (α,β) can be substituted by $(\frac{\pi}{2}, 0)$, which yields the equations:

$$\begin{cases} \dot{x} - l\dot{\theta} = r\dot{\phi}_l \\ \dot{y} = 0 \end{cases}$$

Therefore, we obtain the system for both wheels:

$$\begin{cases} \dot{x} + l\dot{\theta} = r\dot{\phi}_r \\ \dot{x} - l\dot{\theta} = r\dot{\phi}_l \\ \dot{y} = 0 \end{cases}$$

In the case of a straight line, given a start point (x_0, y_0) and an end point (x_1, y_1), the distance to be traveled is L = $\sqrt{(x_1 - x_0)^2 + (y_1 - y_0)^2}$. Also, it is trivial that $\dot{\theta} = 0$. We obtain that $\dot{\phi}_r = \dot{\phi}_l = \frac{\dot{x}}{r}$, for a given forward velocity $\dot{x} = v$ assumed to be constant. Then, by integrating both angular velocities, we obtain the total angular displacement of the motors to be $\Delta\phi = \frac{v\Delta t}{r} = \frac{L}{r}$.

In the case of a rotation about a circle of given radius R, at a constant velocity $\dot{x} = v$, we haven that $\dot{\theta} = \frac{v}{R}$. Solving for wheel angular velocities yields $\dot{\phi}_{l,r} = \frac{v}{r}(1 \pm \frac{l}{R})$ with the car rotating towards the right since it has a smaller angular velocity for a positive R.

b) From sensor readings alone, we only have access to ϕ_r and ϕ_l. At a certain timestep k, we can easily obtain angular velocities through numerical differentiation $\dot{\phi}_k = \frac{\phi_k - \phi_{k-1}}{\Delta t}$, where Δt is the sampling period.

Now, we need to use odometry to keep track of our robot's pose:
First, we obtain the change in wheel angular displacement in order to get the change in wheel COM displacement:

$$\Delta\phi_l = \phi_l - \phi_{l,prev}, \qquad \Delta\phi_r = \phi_r - \phi_{r,prev}$$
$$\Delta s_l = r\Delta\phi_l, \qquad \Delta s_r = r\Delta\phi_r$$

Then, by solving for $\dot{x}$ in the car's kinematic equations, we find that $\dot{x} = \frac{r\dot{\phi}_l + r\dot{\phi}_r}{2}$, and since $\dot{x} \approx \frac{\Delta s}{\Delta t}$, dividing by the period gives us:

$$\Delta s = \frac{r\Delta\phi_l + r\Delta\phi_r}{2} = \frac{\Delta s_l + \Delta s_r}{2}$$

Similarly, solving for $\dot{\theta}$ and dividing by time yields:

$$\Delta\theta = \frac{\Delta s_r - \Delta s_l}{2l}$$

Now, we can update the coordinates of the robot at timestep *k* according to our fixed inertial frame of reference using the following algorithm:

$$\begin{cases} x_k = x_{k-1} + \Delta s \cos(\theta_{k-1} + \frac{\Delta\theta}{2}) \\ y_k = y_{k-1} + \Delta s \sin(\theta_{k-1} + \frac{\Delta\theta}{2}) \\ \theta_k = \theta_{k-1} + \Delta\theta \end{cases}$$

Then, we introduce a change of variables for accurate proportional feedback control given the final point's coordinates (x_f, y_f):

$$\begin{pmatrix} \rho \\ \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} \sqrt{(x - x_f)^2 + (y - y_f)^2} \\ -\theta + atan2(x - x_f, y - y_f) \\ -\theta - \alpha \end{pmatrix}$$

This allows us to introduce the tweakable gains $k_\rho, k_\alpha, k_\beta$ to modify the system's convergence to the final point by updating the following parameters at every iteration:

$$\begin{cases} v = \dot{x} = k_\rho \rho \\ w = \dot{\theta} = k_\alpha \alpha + k_\beta \beta \end{cases}$$

And since $\dot{\theta} = \frac{v}{R}, \dot{x} = v$, we can substitute the new values we have obtained in the following formulas to get the desired motor velocities:

$$\begin{cases} \dot{\phi}_{l,des} = \dfrac{\dot{x}}{r}\left(1 + \dfrac{l\dot{\theta}}{\dot{x}}\right) \\ \dot{\phi}_{r,des} = \dfrac{\dot{x}}{r}\left(1 - \dfrac{l\dot{\theta}}{\dot{x}}\right) \end{cases}$$

Finally, we need to obtain a relation between power commands and desired motor velocities to find the approximate formula:

$$\dot{\phi}_{des} = a\, p_{cmd} + b$$

c) First, we found the following matrix:

$$\begin{pmatrix} a_l & a_r \\ b_l & b_r \end{pmatrix}$$

using the below code:

```
import time
import nxt.locator
from nxt.motor import Motor, Port
import numpy as np
from math import pi

brick = nxt.locator.find()
mL, mR = Motor(brick,Port.A), Motor(brick,Port.B)
tL,tR=[],[]
dt=0.01
for i in (60,65,70,75,80,85,90,95,100):
    mL.run(power=i, regulated=True)
    mR.run(power=i, regulated=True)
    tL.append([])
    tR.append([])
    t0=time.time()
    while time.time()-t0<2:
        if time.time()-t0>0.5:
            tL[-1].append(mL.get_tacho().tacho_count)
            tR[-1].append(mR.get_tacho().tacho_count)
            time.sleep(dt)
mL.brake()
mR.brake()
wL,wR=[],[]
for i in range(len(tL)):
    wLi, wRi = 0, 0
    for j in range(len(tL[i])-1):
        wLi+=(tL[i][j+1]-tL[i][j])/dt
        wRi+=(tR[i][j+1]-tR[i][j])/dt
    wL.append(wLi/(len(tL[i])-1))
    wR.append(wRi/(len(tL[i])-1))

w = np.column_stack([wL,wR])*pi/180
X = np.column_stack([np.arange(60,101,5),np.ones(9)])
ab=np.linalg.pinv(X)@w
```

Then, the main code that implements closed loop control:

```
def wrap(a):
    while a <= -pi: a += 2*pi
    while a >  pi:  a -= 2*pi
    return a

def read_phi_rad(mL=mL, mR=mR):
    phiL = mL.get_tacho().tacho_count * pi/180.0
    phiR = mR.get_tacho().tacho_count * pi/180.0
    return (phiL, phiR)

def update_position(phi,phi_prev,x_prev,r,l):
    dphi=(phi[0]-phi_prev[0],phi[1]-phi_prev[1])
    dsm=(r*dphi[0],r*dphi[1])
    ds=(dsm[0]+dsm[1])/2
    dtheta=(dsm[1]-dsm[0])/(2*l)
    x = (x_prev[0] + ds*cos(x_prev[2]+dtheta/2),
         x_prev[1] + ds*sin(x_prev[2]+dtheta/2),
         wrap(x_prev[2] + dtheta))
    return x

def feedback(x,x_f,r,l,k,ab):
    dx, dy = x_f[0]-x[0], x_f[1]-x[1]
    u = []
    u.append(sqrt(dx**2+dy**2))
    u.append(-x[2]+atan2(dy,dx))
    u.append(-x[2]-u[1])
    v,w = k[0]*u[0], k[1]*u[1]+k[2]*u[2]

    phi_dot_des = ((v+l*w)/r, (v-l*w)/r)
    p_cmd_left = (phi_dot_des[0]-ab[1][0])/ab[0][0]
    p_cmd_right = (phi_dot_des[1]-ab[1][1])/ab[0][1]

    p_cmd_left=np.clip(p_cmd_left,0,100)
    p_cmd_right=np.clip(p_cmd_right,0,100)

    return int(p_cmd_left), int(p_cmd_right), u[0]<0.02
```

```
r, l = 0.026, 0.058
ab = [[0.16202534,0.14453811],[3.730111,4.94268451]]
k=(3.0,8.0,-1.5)

x = (0.0, 0.0, 0.0)
phi_prev = read_phi_rad()
x_f=(1.0,0.0)

dt=0.05
t0 = time.time()
next=t0

while True:
    now = time.time()
    if now < next:
        time.sleep(next-now)
    next +=dt
    phi = read_phi_rad()

    x=update_position(phi,phi_prev,x,r,l)
    phi_prev=phi

    p_cmd_left, p_cmd_right, stop = feedback(x,x_f,r,l,k,ab)
    if stop:
            mL.brake()
            mR.brake()
            break
    mL.run(power=p_cmd_left, regulated=True)
    mR.run(power=p_cmd_right, regulated=True)

    print(f"x={x[0]:.3f} y={x[1]:.3f} th={x[2]:.2f}  pL={p_cmd_left} pR={p
```